

GReinSS

Generative Modeling of Discrete Latent Structures via Dynamic Policy Gradients

Mohammed El-Kebir

University of Illinois Urbana-Champaign

NCI Spring School on Algorithmic Cancer Biology — Tutorial

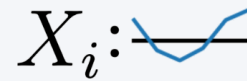
Ivanovic et al., ICML 2026

A recurring statistical inference problem in computational biology

States $S_{1:N} \sim \Pr^*(\mathcal{S})$



Measurements $X_{1:N}$
generated from $S_{1:N}$

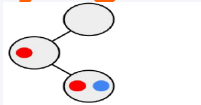


$\mathcal{S}$ is typically **large and combinatorial** — graphs, strings, sets, ...



Rather than directly observing the latent **state** S we care about, we observe some indirect **measurement** X generated from it.

Phylogenies

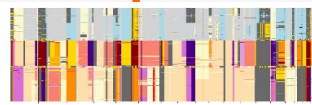


State: tumor evolution tree

Measurement: DNA-seq

[Ivanovic & El-Kebir, RECOMB/Genome Res. 2023]

CNA profiles



State: copy-number profile

Measurement: read depth + BAF

[Ivanovic & El-Kebir, Genome Biol. 2025]

RNA isoforms



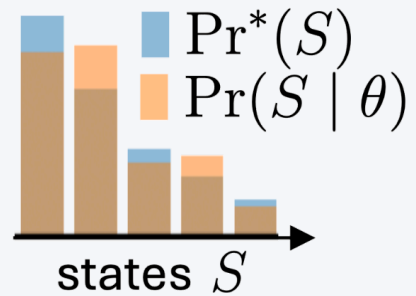
State: spliced transcript

Measurement: aligned short reads

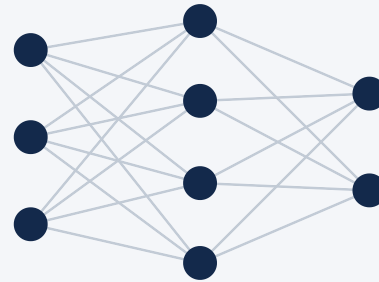
[Ivanovic et al., ICML 2026]

A learning and an inference problem

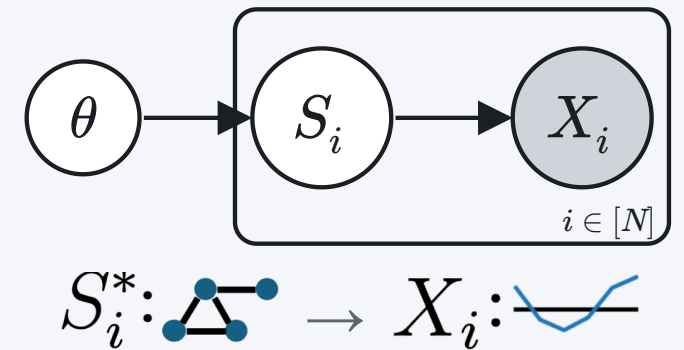
Approximate $\Pr^*(S)$ as
 $\Pr(S \mid \theta)$



Parameters θ : a linear map
or neural network



Generative model with
given $\Pr(X \mid S)$



Problem 1 (Learning). Given (i) $X_{1:N}$ and (ii) $\Pr(X \mid S)$, find θ maximizing
 $\Pr(X_{1:N} \mid \theta) = \prod_i \Pr(X_i \mid \theta)$, where $\Pr(X_i \mid \theta) = \sum_S \Pr(X_i \mid S) \Pr(S \mid \theta)$

Problem 2 (Inference). Given (i) $X_{1:N}$, (ii) $\Pr(X \mid S)$, and (iii) θ , find $\hat{S}_{1:N}$, where
 $\hat{S}_i = \arg \max_S \Pr(X_i \mid S) \Pr(S \mid \theta)$

Why the usual tools struggle

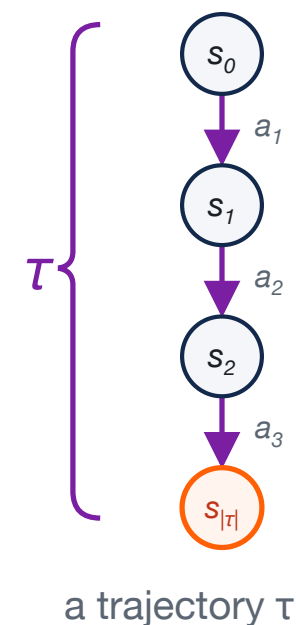
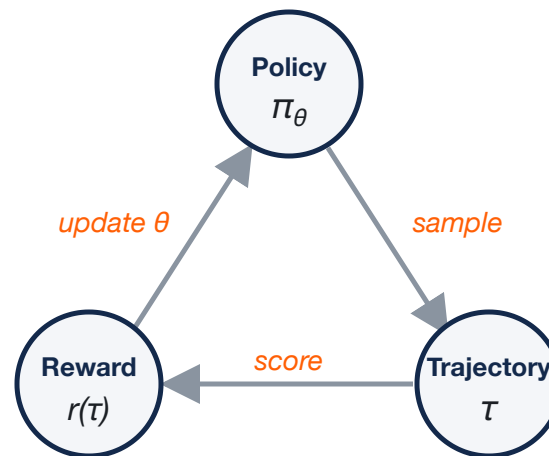
Family	Method	Why it struggles
Exact inference	Expectation–Maximization	E-step exact only for special structure (e.g. HMMs) — intractable for a combinatorial state space
Variational	Variational inference	maximizes an <i>ELBO</i> bound, not the likelihood — needs a tractable posterior over combinatorial states
	Variational autoencoders	learn <i>artificial</i> continuous latents — not the mechanistic state you want
Search	Local search	ignores the shared model across observations
Reinforcement learning	Naive policy gradient	collapses to the single highest-reward state
	GFlowNets	aim to sample in proportion to a fixed reward — not to maximize a likelihood

Gap: none of these directly maximize $\Pr(X_{1:N} \mid \theta)$ over a *discrete, combinatorial* state space.

Primer on reinforcement learning (RL)

Key question: What actions should an agent take to maximize a reward signal?

Concept	In RL
State s_t	current situation
Action a_t	transitions $s_t \rightarrow s_{t+1}$
Policy π_θ	picks the next action
Trajectory τ	path $s_0 \rightarrow \dots \rightarrow s_{ \tau }$
Reward $r(\tau)$	scores terminal state
Objective	$\max_\theta \mathbb{E}_\tau [r(\tau)]$

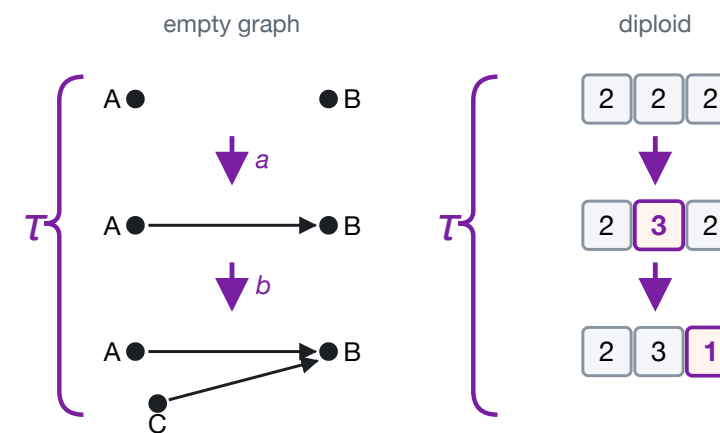


Policy gradient (REINFORCE): $\nabla_\theta \mathbb{E}_{\tau \sim \text{Pr}(\tau|\theta)} [r(\tau)] = \mathbb{E}_\tau [r(\tau) \nabla_\theta \log \text{Pr}(\tau | \theta)]$ –
gradient through $\log \text{Pr}(\tau | \theta)$ only, not the fixed reward $r(\tau)$.

GReinSS: Generative Reinforcement Learning of Structured States

Policy gradient (REINFORCE): $\nabla_{\theta} \mathbb{E}_{\tau \sim \text{Pr}(\tau|\theta)} [r(\tau)] = \mathbb{E}_{\tau} [r(\tau) \nabla_{\theta} \log \text{Pr}(\tau | \theta)]$

Concept	In RL	In GReinSS
State s_t	current situation	<i>same as RL</i>
Action a_t	transitions $s_t \rightarrow s_{t+1}$	<i>same as RL</i>
Policy π_{θ}	picks the next action	<i>same as RL</i>
Trajectory τ	path $s_0 \rightarrow \dots \rightarrow s_{ \tau }$	<i>same, terminal $S(\tau)$</i>
Reward $r(\tau)$	scores terminal state	use $\text{Pr}(X S)$???
Objective	$\max_{\theta} \mathbb{E}_{\tau} [r(\tau)]$	$\max_{\theta} \log \text{Pr}(X_{1:N} \theta)$



Key question: How to set rewards $r(\tau)$ such that

$$\nabla_{\theta} \mathbb{E}_{\tau \sim \text{Pr}(\tau|\theta)} [r(\tau)] = \mathbb{E}_{\tau} [r(\tau) \nabla_{\theta} \log \text{Pr}(\tau | \theta)] = \nabla_{\theta} \log \text{Pr}(X_{1:N} | \theta)?$$

Dynamically-changing rewards

Train with a policy gradient using the **dynamically rescaled reward**

$$r(\tau) = \sum_{i=1}^N \Pr(X_i | \tau) / \Pr(X_i | \theta)$$

$$\nabla_{\theta} \log \Pr(X_{1:N} | \theta)$$

log-likelihood

what we optimize

=

$$\mathbb{E}_{\tau} \left[r(\tau) \nabla_{\theta} \log \Pr(\tau | \theta) \right]$$

policy gradient

how we optimize it

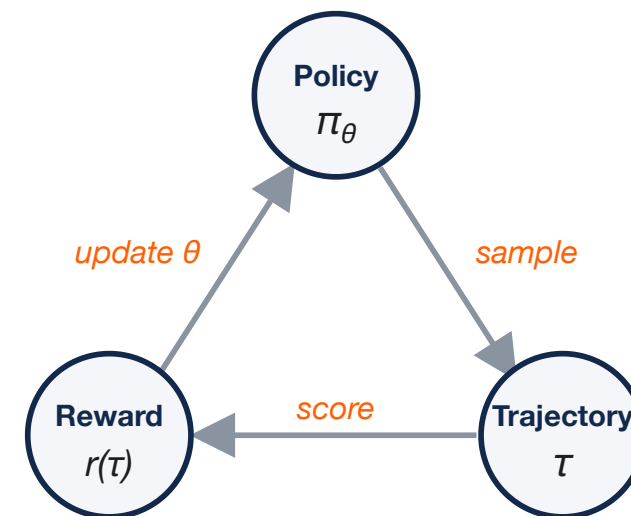
Theorem 1 (Unbiased policy gradient). With the dynamically changing reward $r(\tau) = \sum_{i=1}^N \Pr(X_i | \tau) / \Pr(X_i | \theta)$, the policy gradient $\mathbb{E}_{\tau} [r(\tau) \nabla_{\theta} \log \Pr(\tau | \theta)]$ is an unbiased estimator of $\nabla_{\theta} \log \Pr(X_{1:N} | \theta)$.

GReinSS training loop

The **sample** → **score** → **update** cycle of RL, run with a reward that changes as θ learns:

$$r(\tau) = \sum_{i=1}^N \frac{\Pr(X_i \mid \tau)}{\boxed{\Pr(X_i \mid \theta)}} \quad \Pr(X_i \mid \theta) = \mathbb{E}_{\tau} [\Pr(X_i \mid \tau)]$$

Dynamic reward — the boxed denominator is **re-estimated by sampling** each iteration.



Repeat until convergence:

1. **Sample** a batch $\tau_1, \dots, \tau_M \sim \Pr(\tau \mid \theta)$
2. **Score** each: $\Pr(X_i \mid \tau_j)$
3. **Estimate** $\Pr(X_i \mid \theta) \approx \frac{1}{M} \sum_j \Pr(X_i \mid \tau_j)$ (*same batch*)
4. **Reward** $r(\tau_j) = \sum_i \Pr(X_i \mid \tau_j) / \Pr(X_i \mid \theta)$
5. **Policy-gradient** step along $\mathbb{E}_{\tau} [r(\tau) \nabla_{\theta} \log \Pr(\tau \mid \theta)]$

You supply only two things:

- a **generator** for S (action-by-action)
- the likelihood $\Pr(X \mid S)$

Everything else is generic.

Intuition behind dynamic rewards – why the denominator $\Pr(X_i \mid \theta)$?

$\Pr(X \mid S)$	S_1	S_2	S_3
X_1	.5		
X_2		.3	.2

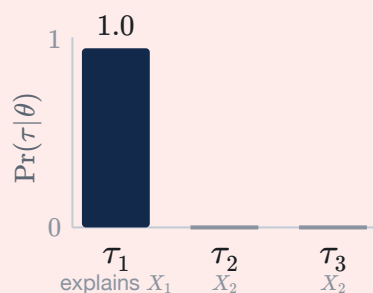
Example: Two measurements X_1 and X_2 . States $\mathcal{S} = \{S_1, S_2, S_3\}$.

Parameters: $\theta \equiv (\Pr(S_1 \mid \theta), \Pr(S_2 \mid \theta), \Pr(S_3 \mid \theta))$

Objective: $\max_{\theta} \Pr(X_1, X_2 \mid \theta) = \max_{\theta} \mathbb{E}_{\tau}[r(\tau)]$

Fixed rewards

$$r(\tau) = \Pr(X_i \mid \tau)$$

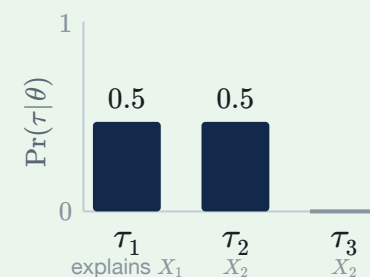


$$\theta^* = (1, 0, 0): \Pr(X_1 \mid \theta) = .5, \Pr(X_2 \mid \theta) = 0$$

$\mathbb{E}_{\tau}[r]$ is linear in the policy $\Rightarrow$ all mass on the best, τ_1
 $\Pr(X_1, X_2 \mid \theta) = .5 \times 0 = 0 \times$

Dynamic rewards

$$r(\tau) = \Pr(X_i \mid \tau) / \Pr(X_i \mid \theta)$$



$$\theta^* = (.5, .5, 0): \Pr(X_1 \mid \theta) = .25, \Pr(X_2 \mid \theta) = .15$$

$\max \mathbb{E}_{\tau}[r] = \max \Pr(X_1, X_2 \mid \theta) \Rightarrow$ balances τ_1, τ_2
 $\Pr(X_1, X_2 \mid \theta) = .25 \times .15 = 0.0375 \checkmark$

→ NOTEBOOK · Demo 1: Set reconstruction

Recover binary **sets** from noisy real-valued measurements. *Trains live in ~10 s.*

Problem. $S_i^* \subseteq \mathcal{U}$, observe
 $X_{i,j} \sim \mathcal{N}(1, \sigma^2)$ if $j \in S_i^*$, else $\mathcal{N}(0, \sigma^2)$.

You'll write:

```
def log_pr_x_given_g(state, obs):  
    return -0.5*np.sum((obs-state)**2)/sigma**2
```

...then `simpleTrainModel(...)`.

Watch for:

- median $\log \Pr(X_i \mid \theta)$ climbing to ~ 0
- recovered sets vs. thresholding the noise
- where the **shared model fixes** noisy bits

Off-policy learning (when on-policy is too slow)

If sampling $\Pr(\tau \mid \theta)$ rarely produces states that explain any X_i , learning stalls.

Sample where the data says to look — bias toward the Bayes posterior:

Theorem 2 (Optimal off-policy proposal). *The unbiased, variance-minimizing sampling proposal is*

$$q(\tau \mid X_{1:N}, \theta) = \frac{1}{N} \sum_{i=1}^N \Pr(\tau \mid X_i, \theta), \quad \Pr(\tau \mid X_i, \theta) = \frac{\Pr(X_i \mid \tau) \Pr(\tau \mid \theta)}{\Pr(X_i \mid \theta)}.$$

Importance sampling keeps the gradient correct. *In cancer apps, a cheap heuristic (e.g. CNNaive in CNRein) seeds plausible states.*

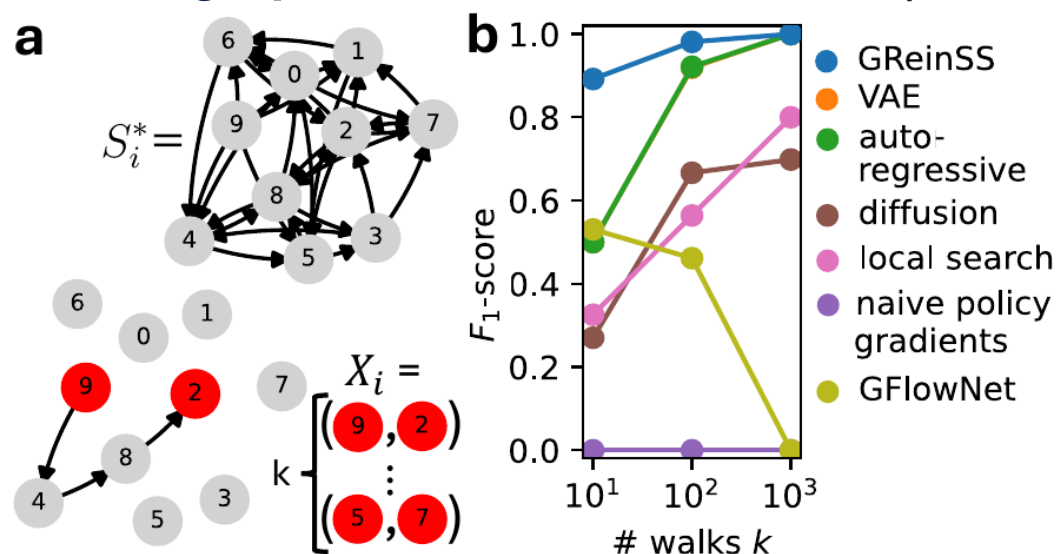
The scoreboard — who actually maximizes the likelihood?

Method	Family	Max. $\prod_i \Pr(X_i \mid \theta)$?
GReinSS	RL	✓ Yes
Naive policy gradient	RL	✗ No
GFlowNets	RL	✗ No
VAE / autoregressive / diffusion + EM	Variational · EM	≈ approx.
Local search	Search	✗ No

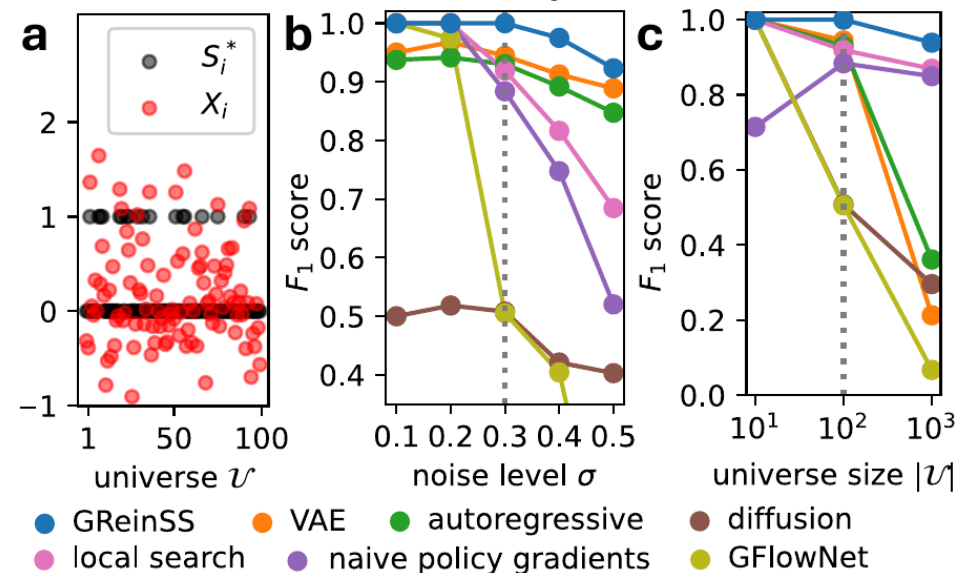
GFlowNets — closest cousin, different goal: they *sample* states $\propto$ a **fixed, given reward** $R(S)$ (diverse candidates); GReinSS has **no fixed reward**, *rescaling* it each step so maximizing it **provably equals maximum-likelihood** learning.

Results — simulations

Latent graphs from random-walk endpoints



Latent sets from noisy measurements



$k = 10$ walks: GReinSS $F_1 = \mathbf{0.891}$; all baselines < 0.55

$|\mathcal{U}| = 1000$: GReinSS $F_1 = \mathbf{0.938}$; GEM baselines < 0.4

Dynamic rewards are a **small** code change over naive PG — but decisive. Naive PG here predicts the **empty graph** ($F_1 = 0$).

→ NOTEBOOK · Demo 2: Graph inference (pre-trained)

Latent **directed graphs** from start/end points of k absorbing random walks.

State = 90 possible directed edges (10 nodes).

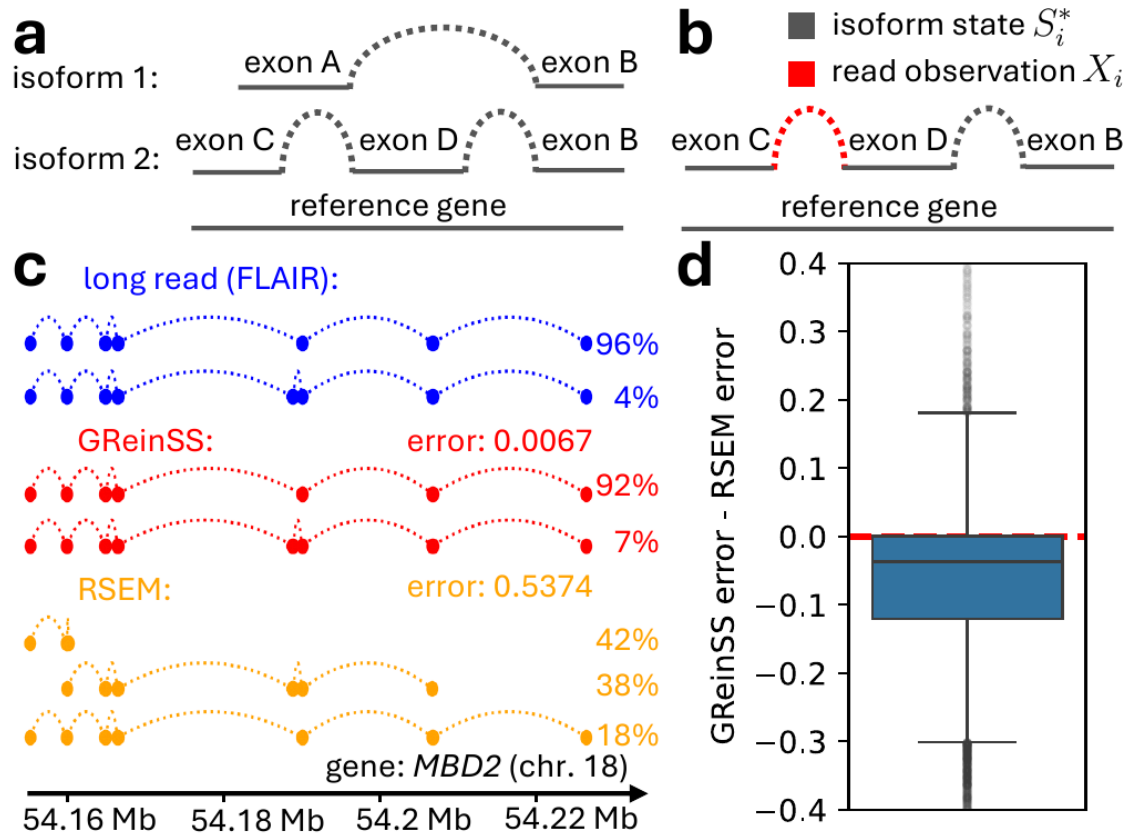
$\Pr(X \mid S)$ from the shifted-Laplacian $(L + I)^{-1}$ random-walk model.

We **load a pre-trained policy**, run inference, and score edge-recovery F_1 against ground-truth graphs.

Watch for:

- training likelihood curve (pre-computed)
- a reconstructed graph $\hat{S}_i$ vs. true S_i^*
- per-graph F_1 distribution

Application — RNA isoforms beat RSEM



State S = an isoform (chosen exon junctions) + sample + read position.

$\Pr(X | S) = 1$ iff read position & sample match — trivial forward model.

On **GTEx** (61 samples w/ matched long reads, 14,390 genes):

- GReinSS **beats RSEM by ≥ 0.05** on **46.6%** of genes; RSEM beats GReinSS on only **9.4%**
- *MBD2* example: GReinSS error **0.007** vs RSEM **0.537**

GReinSS already powers two cancer methods

CloMu

Tumor **phylogenies** of SNVs

States: mutation trees

Observations: noisy DNA-seq trees

Ivanovic & El-Kebir, RECOMB / Genome Res. 2023

CNRein

Copy-number evolution in single cells

States: sets of CNA events per cell

Observations: single-cell DNA-seq

Ivanovic & El-Kebir, Genome Biol. 2025

This paper **generalizes** the shared technique behind both into one framework for *any* discrete latent structure — with theory, off-policy theory, and baselines.

When should *you* reach for GReinSS?

Good fit ✓

- latent state is **discrete** / **combinatorial**
- you can **generate** it incrementally
- you know (or can compute) $\Pr(X \mid S)$
- observations are **indirect** & shared structure exists

Reach elsewhere ✗

- continuous latents → VAE / diffusion
- tractable exact E-step → classic EM
- rewards truly fixed & known → standard RL / GFlowNet

Recipe: ① write a generator for S · ② write $\Pr(X \mid S)$ · ③ `simpleTrainModel` · ④ `simpleInference` . *(optional)* add an off-policy proposal for hard instances.

Thank you — let's build

Code + notebook: `code/README.md`, `tutorial/GReinSS_demo.ipynb`

Recipe: generator for S + likelihood $\Pr(X \mid S) \rightarrow \text{train} \rightarrow \text{infer}$

Questions? · melkebir@illinois.edu